

NYCU Computer Vision 2026 HW2

111550005 林均豪

1. Introduction

In this assignment, the task is to detect digits (0–9) in RGB images, predicting the bounding box and class label of each digit. The dataset contains 30,062 training images, 3,340 validation images, and 13,068 test images, with annotations provided in COCO format.

I implemented Deformable DETR with a ResNet-50 backbone. Deformable DETR improves upon the original DETR by replacing global attention with multi-scale deformable attention, which significantly reduces convergence time and improves detection of small objects, both critical properties for digit detection.

The model achieved a validation mAP of 0.4342 by epoch 12, with leaderboard score of 0.35.

2. Method

2.1 Data Preprocessing

All images are resized to 256×256 using letterbox resizing (aspect-ratio-preserving resize with grey padding at value 114), which avoids distortion and preserves the spatial layout of digits.

For training data, the following augmentation pipeline is applied using Albumentations:

- OneOf: GaussNoise / GaussianBlur (3×5) / MotionBlur (3×5) with probability 0.4 — simulates camera quality variation
- ColorJitter: brightness=0.3, contrast=0.3, saturation=0.2, hue=0.05 with probability 0.5
- Affine scale jitter: scale in [0.85, 1.15] with probability 0.5 — light scale variation without rotation or flipping

Note: Horizontal flipping is intentionally excluded because flipping changes the identity of certain digits (e.g., 6 ↔ 9).

All images are normalized with ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225].

2.2 Model Architecture and Hyperparameters:

The model follows the Deformable DETR architecture (Zhu et al., 2021) with the following components:

- **Backbone:** ResNet-50 pretrained on ImageNet (IMAGENET1K_V2). All BatchNorm layers are frozen throughout training. Four feature levels C2–C5 are extracted at strides 4, 8, 16, 32.
- **Feature Projection:** Each level is projected to 256 dimensions via a 1×1 Conv + GroupNorm(32) layer. A learnable level embedding is added to distinguish different scales.
- **Encoder:** 6 layers of deformable self-attention + FFN. Each query predicts sampling offsets over all feature levels.
- **Decoder:** 6 layers with standard self-attention followed by deformable cross-attention. Implements iterative box refinement: each decoder layer predicts a residual relative to the previous layer's reference points.
- **Heads:** Each decoder layer has its own classification head (Linear $\rightarrow$ 11 classes including background) and box head (3-layer MLP $\rightarrow$ 4 values). Only the last layer's output is used during inference.
- **Object Queries:** 100 learnable queries, each split into positional and content components.

2.3 Hyperparameters

Parameter	Value
Image size	256×256
Batch size	4
Total epochs	50
Peak LR (transformer)	$1e-4$
Backbone LR	$1e-5$
Weight decay	$1e-4$

Warmup epochs	5
LR schedule	Linear warmup + Cosine decay
Mixed precision	bfloat16 (bf16)
Gradient clip	0.1
Number of queries	100
Encoder / Decoder layers	6 / 6
FFN hidden dim	512
Deformable attention points	2 per level

2.4 Loss Function

The set prediction loss consists of three components, with auxiliary losses computed at every decoder layer (weighted by 0.5):

- Classification Loss: Cross-entropy with class weights [1.0, ..., 1.0, 0.1] where the background class is down-weighted.
- L1 Loss: L1 distance between predicted and GT bounding boxes in normalized cxcywh format (weight = 5.0).
- GIoU Loss: Generalized IoU loss for better localization (weight = 2.0).

Hungarian matching is used to find the optimal one-to-one assignment between predictions and GT boxes, minimizing the combined matching cost.

2.5 Training Strategy

The optimizer is AdamW with separate learning rates for backbone and non-backbone parameters. The learning rate follows a linear warmup over 5 epochs, then cosine decay to zero. bfloat16 mixed precision is used to reduce memory usage and speed up training.

Checkpoints are saved every epoch (last.pth) and whenever a new best validation mAP is achieved (best.pth), ensuring training can be resumed after interruption.

3. Results

Epoch	Train Loss	Val mAP
0	4.99	0.311
3	3.42	0.391
6	3.20	0.382
9	3.08	0.411
12	2.99	0.434

The model converged quickly — by epoch 3 it already exceeded the strong baseline (0.38).

The validation mAP continued to improve steadily throughout training.

Leaderboard Score:0.35

690043	111550005	0.35
--------	-----------	------

4. References

Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2021). Deformable DETR: Deformable Transformers for End-to-End Object Detection. ICLR 2021. <https://arxiv.org/abs/2010.04159>

Carion, N., et al. (2020). End-to-End Object Detection with Transformers. ECCV 2020.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.

5. Additional Experiments

Experiment: Deformable vs. Standard DETR

Hypothesis: Standard DETR requires hundreds of epochs to converge due to global attention over all spatial locations. Replacing it with multi-scale deformable attention should drastically

reduce convergence time.

I initially implemented standard DETR (6-layer encoder/decoder, full attention). After 10 epochs, validation mAP remained near 0 because the model had not converged yet. Switching to Deformable DETR achieved mAP=0.311 by the end of epoch 0, confirming that deformable attention converges much faster.

Result: Deformable DETR reached strong baseline within 3 epochs; standard DETR did not converge within 10 epochs.

6. Code Reliability

```
PS C:\Users\user\Desktop\DL\HW2> flake8 train.py
PS C:\Users\user\Desktop\DL\HW2> █
```

GitHub: <https://github.com/swimming-in-cs/NYCU-Computer-Vision-2026-HW2>